

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

Projekt iz kolegija Bioinformatika 1, 2025./2026.

Službena stranica: <https://www.fer.unizg.hr/predmet/bio1>

Bamboo filter

Projektni rad

Autori:

Ana Angelov
Hana Dolovčak

Zagreb, svibanj 2026.

Sadržaj

1	Uvod	2
2	Pregled pozadinskih struktura	2
2.1	Bloomov filter	2
2.2	Cuckoo filter	3
3	Bamboo filter	4
3.1	Četveroslojni model pohrane	4
3.2	Linearno raspršivanje s djelomičnim ključem	5
3.3	Dijeljeni indeksni otisak	6
3.4	Vizualizacija na primjeru	7
3.5	Pseudokod operacija	8
4	Implementacija	8
4.1	Pregled arhitekture	8
4.2	Klasa Segment	9
4.3	Klasa BambooFilter	10
4.4	Hashiranje k-mera	11
4.5	Izgradnja i pokretanje	11
5	Eksperimentalna evaluacija	12
5.1	Postav eksperimenta	12
5.2	Sintetički podaci	12
5.3	E. coli genom	14
5.4	Usporedba s referentnom implementacijom	15
5.5	Analiza rezultata	16
6	Zaključak	17

1 Uvod

Kada sustav treba brzo provjeriti pripada li određeni element velikom skupu podataka, pohrana cijelog skupa u radnu memoriju često nije izvediva. Za takve se scenarije koriste strukture podataka za aproksimativne upite o članstvu (engl. *approximate membership query*, AMQ). To su probabilističke strukture koje odgovaraju na upit o članstvu uz prihvatljivu stopu lažno pozitivnih odgovora, ali uz znatno manju memorijsku potrošnju. Najčešće korišteni primjer je Bloomov filtar [5], a *Cuckoo* filtar [3] proširuje tu ideju podrškom za brisanje elemenata i većom propusnošću pretraživanja.

Zajednički nedostatak takvih struktura je ograničena podrška za *dinamičku promjenu veličine* (engl. *resizing*). Kada broj elemenata značajno poraste, filtar mora proširiti svoj kapacitet. Postojeća rješenja svode se na dva pristupa, od kojih svaki ima određene nedostatke. Pristup koji ponovno raspršuje (*rehashira*) sve elemente u novu tablicu jamči dobre performanse, ali blokira ostale operacije dok god taj postupak traje. Pristup koji dodaje nove filtre u popis izbjegava blokiranje, ali svako pretraživanje mora provjeriti sve filtre u popisu, pa propusnost pada s brojem proširenja. *Dynamic Cuckoo* filtar (DCF) [6] na taj način gubi 70,67% propusnosti pretraživanja već nakon tri proširenja [2].

Bamboo filtar [1, 2] rješava oba problema tako da pri svakom proširenju ponovno raspršuje samo jedan segment tablice, a ne cijelu strukturu. Time se trajanje blokiranja svodi na minimum, a složenost pretraživanja ostaje $\mathcal{O}(1)$ jer svaki element uvijek pripada točno jednom segmentu, bez obzira na broj prethodnih proširenja.

U ovom projektu implementiramo *Bamboo* filtar u programskom jeziku C++17 i vrednujemo ga na skupovima k -mera izvučenih iz genoma bakterije *Escherichia coli* K-12 te iz sintetički generiranih DNA nizova, za $k \in \{10, 20, 50, 100, 200\}$. Implementaciju uspoređujemo s originalnom [11] po propusnosti umetanja i pretraživanja, memorijskoj potrošnji i stopi lažno pozitivnih odgovora.

2 Pregled pozadinskih struktura

2.1 Bloomov filtar

Bloomov filtar [5] jedna je od najstarijih i najraširenijih probabilističkih struktura podataka. Temelji se na polju bitova duljine m i skupu od k neovisnih funkcija za raspršivanje. Svaki element koji se umeće raspršuje se pomoću svih k funkcija, a dobiveni se indeksi koriste za postavljanje odgovarajućih bitova u polju na vrijednost 1. Pretraživanje elementa provodi se analogno: provjerava se svih k bitova i, ako je barem jedan jednak nuli, element sigurno nije u skupu. Ako su svi jednaki jedan, element je *vjerojatno* u skupu, no postoji mogućnost lažno pozitivnog odgovora jer su ti bitovi mogli biti postavljeni pri

umetanju nekih drugih elemenata.

Bloomov filtar nikada ne daje lažno negativne odgovore. Lažno pozitivni odgovori su mogući i njihova stopa ε ovisi o duljini polja bitova m , broju pohranjenih elemenata n i broju funkcija za raspršivanje k . Za optimalan odabir $k = \ln 2 \cdot m/n$, stopa lažno pozitivnih odgovora iznosi:

$$\varepsilon \approx \left(1 - e^{-kn/m}\right)^k. \quad (1)$$

Postizanje manje stope lažno pozitivnih odgovora zahtijeva dulje polje bitova ili više funkcija za raspršivanje. Oboje povećava memorijsku potrošnju ili broj potrebnih operacija po upitu. Važno ograničenje Bloomova filtra je što ne podržava brisanje jer bi brisanje bita postavljenog za jedan element moglo narušiti informaciju o drugom elementu koji dijeli isti bit. Osim toga, dinamička promjena veličine strukture zahtijeva izgradnju potpuno novog filtra i ponavljanje postupka umetanja za sve elemente.

2.2 Cuckoo filtar

Cuckoo filtar [3] naslanja se na ideju *kukavičjeg raspršivanja* [10], tehnike otvorenog adresiranja u kojoj svaki element ima točno dva kandidatska mjesta u tablici. Za razliku od Bloomova filtra, koji izravno radi s originalnim elementima, *Cuckoo* filtar pohranjuje samo kratke *otiske*, odnosno kratke raspršene reprezentacije elemenata fiksne duljine (engl. *fingerprint*). Tablica se sastoji od niza pretinaca, pri čemu svaki pretinac može pohraniti b otisaka.

Umetanje. Za svaki element x izračunava se otisak $f = \text{hash}(x)$ i dva kandidatska pretinca:

$$h_1(x) = \text{hash}(x), \quad h_2(x) = h_1(x) \oplus \text{hash}(f). \quad (2)$$

XOR operacija u jednadžbi (2) osigurava da iz bilo kojeg od dva pretinca možemo izračunati alternativni pretinac koristeći samo pohranjeni otisak, bez poznavanja originalnog elementa:

$$j = i \oplus \text{hash}(f). \quad (3)$$

Ovo svojstvo naziva se *kukavičje raspršivanje s djelomičnim ključem* i ključno je za mogućnost premještanja otisaka pri umetanju. Ako su oba kandidatska pretinca puna, jedan se pohranjeni otisak istiskuje na svoju alternativnu poziciju (analogno ponašanju ptice kukavice koja izbacuje tuđa jaja iz gnijezda), a postupak se ponavlja. Ako istiskivanje ne uspije unutar zadanog maksimalnog broja koraka, filtar se smatra punim.

Pretraživanje i brisanje. Pretraživanje provjerava oba kandidatska pretinca; ako otisak nije pronađen ni u jednom, element sigurno nije u skupu. Za brisanje je dovoljno pronaći i ukloniti jedan primjerak otiska iz jednog od pripadajućih pretinaca. Ovo je značajna prednost u usporedbi s Bloomovim filtrom koji operaciju brisanja ne podržava.

Prostorna učinkovitost. Pohranom isključivo otisaka (umjesto punih ključeva) i visokom popunjenošću tablice (do 95% uz $b = 4$), *Cuckoo* filter postiže bolju prostornu učinkovitost od Bloomova filtra za stope lažno pozitivnih odgovora ispod 3%. Duljina otiska f izravno određuje stopu lažno pozitivnih odgovora. Za pretinac veličine b vrijedi:

$$\varepsilon \approx \frac{2b}{2f}. \quad (4)$$

Osnovni nedostatak *Cuckoo* filtra je fiksni kapacitet. Kada tablica dosegne granicu popunjivosti, umetanja počinju neuspijevati, a struktura se mora u potpunosti rekonstruirati, što blokira sve operacije za vrijeme trajanja rekonstrukcije.

3 Bamboo filter

Bamboo filter [1, 2] rješava navedene probleme uvođenjem *segmentne strukture* koja omogućuje promjenu veličine u malim, kontroliranim koracima. U nastavku su opisani ključni mehanizmi te strukture.

3.1 Četveroslojni model pohrane

Bamboo filter organizira podatke u četiri razine hijerarhije:

$$\text{tablica raspršivanja} \supset \text{segmenti} \supset \text{pretinci} \supset \text{utori}$$

Svaki utor pohranjuje jedan otisak. Više utora čini jedan pretinac; više uzastopnih pretinaca čini jedan segment, a svi segmenti zajedno čine tablicu raspršivanja. Svaki segment predstavlja zaseban *Cuckoo* filter, to jest samostalnu tablicu s mehanizmom kukavičjeg istiskivanja. Struktura je ime dobila upravo po vizualnom izgledu – niz segmenata podsjeća na bambus.

Projektna odluka je da segment bude najmanja jedinica promjene veličine. Svako proširenje dodaje točno jedan novi segment i ponovno raspršuje (*rehašira*) isključivo elemente tog segmenta. Takva granularnost donosi čak tri prednosti:

Trajanje blokiranja je kratko i predvidivo jer se u jednom koraku obrađuje samo jedan segment fiksne veličine, neovisno o ukupnoj veličini tablice. Veličina segmenta može se koristiti kao parametar za kontrolu maksimalnog trajanja blokiranja.

Prostor raste postupno jer se dodaje samo jedan segment po proširenju. Kod tradicionalnog udvostručavanja tablice, neposredno nakon proširenja čak polovica memorijskog prostora ostaje neiskorištena.

Lokalnost priručne memorije (cache locality) poboljšana je jer su pretinci unutar jednog segmenta smješteni sekvencijalno u memoriji, pa se čitav segment može učitati u priručnu memoriju u jednom ili nekoliko pristupa radnoj memoriji.

3.2 Linearno raspršivanje s djelomičnim ključem

Bamboo filtar uvodi tehniku *linearnog raspršivanja s djelomičnim ključem* (engl. *partial-key linear hashing*) za organizaciju segmenata. Za razliku od DCF-a koji organizira filtre u popis i mora ih sekvencijalno provjeravati, *Bamboo* filtar uvijek u *točno koji segment* pojedini element pripada.

Segmentima se mijenja veličina u *logičkom redosljedu*: od prvog prema zadnjem, krug za krugom. Nakon što se svi segmenti prošire u jednom krugu, započinje novi krug, a kapacitet tablice se udvostručuje. Jasno razgraničenje između proširenog i izvornog dijela tablice omogućuje algoritmu da u svakom trenutku izravno izračuna točan indeks segmenta kojemu element pripada.

Formalno, za svaki element e s vrijednošću raspršivanja $h(e)$ izračunavaju se otisak, primarni i sekundarni indeks pretinca iz *iste* vrijednosti raspršivanja:

$$f(e) = (h(e) \gg (l_b + l_{s_0})) \bmod 2^{l_f}, \quad (5)$$

$$I_b(e) = h(e) \bmod 2^{l_b}, \quad (6)$$

$$I'_b(e) = (I_b(e) \oplus f(e)) \bmod 2^{l_b}, \quad (7)$$

gdje l_b označava broj bitova za indeks pretinca, l_{s_0} početni broj bitova za indeks segmenta, a l_f broj bitova otiska. Sekundarni pretinac računa se XOR operacijom analogno jednadžbi (3) iz *Cuckoo* filtra, omogućujući kukavičje istiskivanje bez poznavanja originalnih ključeva.

Indeks segmenta za element koji je prošao $(i + 1)$ -ti krug proširenja iznosi:

$$I_s(e) = (h(e) \gg l_b) \bmod (2^{i+1} N_0), \quad (8)$$

pri čemu je N_0 početni broj segmenata. Ako izračunati indeks premašuje trenutnu veličinu tablice, primjenjuje se korekcija oduzimanjem vrijednosti $2^i N_0$. Svako pretraživanje

zahtijeva samo jedan izračun segmentnog indeksa i provjeru jednog segmenta. Složenost pretraživanja iznosi $\mathcal{O}(1)$ neovisno o broju prethodnih proširenja.

Segmenti koji su prekoračili kapacitet, a još nisu prošireni u tekućem krugu, privremeno pohranjuju višak elemenata u preljevni lanac. Taj se lanac (popis) isprazni čim segment prođe postupak proširenja. Pretraživanje provjerava i preljevni lanac uz osnovni segment.

3.3 Dijeljeni indeksni otisak

Pri proširenju segmenta potrebno je znati u koji novi segment treba premjestiti svaki element. U standardnom Cuckoo filteru to je nemoguće jer su pohranjeni samo fingerprinti, a hash funkcija je jednosmjerna – iz fingerprinta se ne može rekonstruirati originalni element.

Bamboo filtar rješava taj problem tehnikom *index-shared fingerprint*. Mehanizam se temelji na djelomičnom preklapanju bitova unutar 64-bitne hash vrijednosti, pri čemu fingerprint i indeks segmenta neovisno koriste isti skup bitova višeg reda.

Kada se tablica nalazi u početnom stanju, segmentni indeks I_s koristi l_{s_0} bitova smještenih neposredno iznad indeksa pretinca, dok se otisak izračunava iz l_f bitova iznad te granice, sukladno jednadžbama (5) i (8). Međutim, sa svakim novim krugom proširenja, broj bitova potrebnih za adresiranje segmenata (l_s) raste. Budući da se bitovi za I_s uzimaju s iste pozicije prema bitovima veće važnosti, indeks segmenta s porastom kapaciteta počinje obuhvaćati najniže bitove pohranjenog otiska.

Prilikom proširenja, algoritam identificira taj točno određeni zajednički bit unutar samog otiska: ako je taj bit jednak 1, element se premješta u novododijeljeni segment, a u suprotnom ostaje u postojećem. Time je omogućena precizan prijenos podataka bez potrebe za ponovnim računanjem hash-a iz originalnog ključa.

Važno je napomenuti da prijenos jednog bita iz otiska u segmentni indeks ne povećava stopu lažno pozitivnih odgovora. Smanjenje razlikovne moći fingerprinta nadoknađuje se povećanjem segmentnog indeksa, a ukupna količina informacije kojom se elementi razlikuju ostaje ista. Porast stope lažno pozitivnih koji se ipak javlja s brojem proširenja posljedica je ukupnog povećanja broja elemenata, a ne gubitka informacije, i može se izraziti kao [2]:

$$\varepsilon(z) \approx \frac{\lceil 2b + 2bl(z) \rceil}{2^{l_f - n_e}}, \quad (9)$$

gdje je b broj utora po pretincu, $l(z)$ očekivana duljina preljevnog lanca pri faktoru opterećenja z , a n_e broj provedenih krugova proširenja.

3.4 Vizualizacija na primjeru

Razmatramo filter s $N_0 = 2$ segmenta, 4 pretinca po segmentu, 1 utorom po pretincu i $l_f = 3$ bita otiska ($l_b = 2, l_{s_0} = 1$). Inicijalno je filter prazan:

	bucket 00	bucket 01	bucket 10	bucket 11
Segment 0	–	–	–	–
Segment 1	–	–	–	–

Umetanje. Za element čija je hash vrijednost 101101 ($l_b = 2, l_{s_0} = 1, l_f = 3$):

- indeks pretinca: 01 (niža 2 bita),
- segment indeks: 1 (sljedeći bit),
- otisak: 101 (viša 3 bita).

Otisak 101 pohranjuje se u pretinac 01 segmenta 1. Sekundarni pretinac računa se kao $I'_b = 01 \oplus \text{hash}(101)$, i pri eventualnoj kukavičje istiskivanju može se koristiti taj alternativni pretinac unutar istog segmenta.

Proširenje prvog kruga. Kada filter dosegne prag opterećenja, segment 0 je sljedeći na redu za dijeljenje. Dodaje se novi segment 2. Svaki otisak iz segmenta 0 promatra se po bitu 0 (odgovara prvom krugu proširenja):

Segment	Sadržaj nakon ekspanzije
Segment 0	zadržava otiske čiji je bit 0 = 0
Segment 2	prima otiske čiji je bit 0 = 1
Segment 1	nepromijenjen (još čeka na red)

Budući da su otisci nasumični, otprilike pola odlazi u novi segment. Kada se i segment 1 podijeli (sljedeće proširenje u istom krugu), filter ima 4 segmenta i krug proširenja je završen. Sljedeći krug počinje ispočetka od segmenta 0, ali sada promatra bit 1 otiska.

Pretraživanje. Za svaki upit izračunava se I_s prema jednadžbi (8) i provjerava se samo odgovarajući segment (te eventualni preljevni lanac). Bez obzira na to koliko je proširenja prethodno provedeno, pretraživanje pristupa uvijek točno jednom segmentu.

3.5 Pseudokod operacija

Algorithm 1 Insert(e)

```
1: Izračunaj  $f$ ,  $I_b$ ,  $I'_b$ ,  $I_s$  prema jednadžbama (5)–(8)
2: if bucket  $I_b$  ili  $I'_b$  u segmentu  $I_s$  ima prazan slot then
3:   Pohrani  $f$ ; return
4: end if
5: for  $kick = 0$  to  $M_e$  do
6:   Nasumično iseli fingerprint iz trenutnog bucketa (kukavičje istiskivanje)
7:   if alternativni pretinac ima slobodan slot then return
8:   end if
9: end for
10: Pohrani  $f$  u preljevni lanac segmenta  $I_s$ 
11: if uvjet za ekspanziju je ispunjen then pozovi EXTEND()
12: end if
```

Algorithm 2 Extend()

```
1:  $p \leftarrow$  indeks sljedećeg segmenta za dijeljenje
2: Dodaj novi segment u hash tablicu
3: for all fingerprint  $f$  u segmentu  $p$  do
4:   if bit  $round$  od  $f$  je 1 then premjesti  $f$  u novi segment
5:   end if
6:   Inkrementiraj  $p$ ; ako su svi segmenti prošli, povećaj krug ekspanzije
```

Algorithm 3 Pretraži(e)

```
1: Izračunaj  $f$ ,  $I_b$ ,  $I'_b$ ,  $I_s$ 
2: if bucket  $I_b$  ili  $I'_b$  u segmentu  $I_s$  sadrži  $f$  then return true
3: end if
4: if preljevni lanac segmenta  $I_s$  nije prazan then pretraži preljevni lanac
5: end if
6: return false =0
```

4 Implementacija

4.1 Pregled arhitekture

Implementacija je organizirana u tri glavna modula unutar direktorija `src/`. Modul `segment.{h,cpp}` implementira klasu `Segment`, cuckoo hash tablicu unutar jednog segmenta (autorica: Hana Dolovčak). Modul `bamboo_filter.{h,cpp}` implementira klasu

BambooFilter koja upravlja radom segmenata, upravlja partial-key linear hashingom i index-shared fingerprintom (autorica: Ana Angelov). Modul `hash.{h,cpp}` nudi hashiranje k -mera putem `ntHash` [9] i pomoćne hash operacije (autorica: Ana Angelov).

Direktorij `benchmark/` sadrži CLI alat koji mjeri propusnost i memorijsku potrošnju, analizator FASTA datoteka, generator sintetičkih DNA nizova i komponenta za izdvajanje k -mera. Svi parametri unose se isključivo iz naredbenog retka. Autorice su Hana Dolovčak (analizator, timer, generator, komponenta za izdvajanje k -mera) i Ana Angelov (glavni program za ispitivanje performansi). Direktorij `extern/` sadrži `reference_adapter.cpp`, adapter koji omotava originalnu implementaciju [11] u isti CLI i CSV format radi izravne usporedbe (autorica: Ana Angelov). Jedinični testovi u `test/` pokrivaju sve module (autorice: Ana Angelov za `test_bamboo`, Hana Dolovčak za ostale).

4.2 Klasa Segment

Svaki utor pohranjen je kao vrijednost tipa `uint16_t`: bit 15 (`kOccupiedBit = 0x8000`) označava da je utor zauzet, a preostalih do 15 bita pohranjuje otisak.

```
1 bool Segment::try_place(size_t bucket, uint32_t fp) {
2     uint16_t encoded = static_cast<uint16_t>(kOccupiedBit | (fp &
3         fp_mask_));
4     for (size_t s = 0; s < bucket_size_; ++s) {
5         if ((read_slot(bucket, s) & kOccupiedBit) == 0) {
6             write_slot(bucket, s, encoded);
7             ++occupancy_;
8             return true;
9         }
10    }
11    return false;
12 }
```

Listing 1: Umetanje otiska u utor unutar pretinca

Alternativni pretinac izračunava se XOR-om kao u originalnom Cuckoo filteru [3], omogućujući pronalazak alternativnog pretinca koristeći samo pohranjeni fingerprint:

```
1 size_t Segment::alt_bucket(size_t bucket, uint32_t fp) const {
2     // h2 = h1 XOR mini_hash(fp), dakle vrijedi i obrnuto
3     return bucket ^ (mini_hash(fp) & bucket_mask_);
4 }
```

Listing 2: Izračun alternativnog pretinca bez originalnog ključa

Kada kukavičje istiskivanje ne uspije unutar `kMaxKicks = 500` koraka, element se privremeno pohranjuje u `overflow_` vektor koji implementira preljevni lanac segmenta.

4.3 Klasa `BambooFilter`

Metoda `decompose()` implementira partial-key linear hashing iz jednadžbi (5)–(8). Ključni dio je određivanje kojemu segmentu element pripada – tablica se dijeli na segmente koji su već prošli ekspanziju tekućeg kruga i one koji još čekaju:

```
1 void BambooFilter::decompose(uint64_t h, size_t& seg,
2                               size_t& bucket, uint32_t& fp) const {
3     bucket = static_cast<size_t>(h & ((1ULL << l_b_) - 1));
4     fp = fingerprint_bits(h, l_b_ + l_s0_, config_.fp_bits);
5
6     size_t small_count = (1ULL << expand_round_) *
7                           n_initial_segments_;
8     size_t large_count = small_count << 1;
9     size_t cand = static_cast<size_t>((h >> l_b_) & (large_count -
10                                     1));
11
12     if (cand < small_count) {
13         seg = cand;
14     } else {
15         size_t offset = cand - small_count;
16         seg = (offset < split_pointer_) ? cand : offset;
17     }
18 }
```

Listing 3: Dekompozicija hash vrijednosti na fingerprint, bucket i segment indeks

Metoda `extend()` izdvaja elemente čiji je odgovarajući bit fingerprinta jednak 1 u novi segment, implementirajući tehniku index-shared fingerprint:

```
1 void BambooFilter::extend() {
2     size_t p = split_pointer_;
3     auto drained = segments_[p]->drain_entries_with_fp_bit(
4         expand_round_, /*bit_value=*/true);
5     auto drained_overflow = segments_[p]->
6         drain_overflow_with_fp_bit(
7             expand_round_, /*bit_value=*/true);
8
9     segments_.push_back(std::make_unique<Segment>(...));
10    for (const auto& e : drained)
```

```

10     segments_.back()->insert(e.fp, e.bucket);
11     for (const auto& e : drained_overflow)
12         segments_.back()->insert(e.fp, e.bucket);
13
14     ++split_pointer_;
15     if (split_pointer_ >= small_count) {
16         split_pointer_ = 0;
17         ++expand_round_;
18     }
19 }

```

Listing 4: Ekspanzija jednog segmenta

4.4 Hashiranje k-mera

Za hashiranje k-mera koristi se **ntHash** [9], rekurzivna hash funkcija dizajnirana za DNA sekvence. Prednost ntHasha je *klizno raspršivanje* (engl. *texttrolling hash*): pomak prozora za jedan nukleotid zahtijeva svega dvije bitne operacije umjesto ponovnog hashiranja cijelog k-mera značajno ubrzavajući izdvajanje k-mera iz dugačkih sekvenci.

```

1 // Inicijalizacija za prvi k-mer
2 fh_val_ = NTF64(seq_, static_cast<unsigned>(k_));
3
4 // Pomak prozora: uklanjamo char_out, dodajemo char_in
5 fh_val_ = NTF64(fh_val_, k_, char_out, char_in);

```

Listing 5: Rolling hash – pomak prozora za jedan nukleotid

4.5 Izgradnja i pokretanje

```

1 # Konfiguracija i izgradnja
2 cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
3 cmake --build build -j 4
4
5 # Jedini ni testovi
6 cd build && ctest --output-on-failure
7
8 # Sintetički benchmark (n u {10^3..10^7}, k u {10,20,50,100,200})
9 ./scripts/run_synthetic.sh
10
11 # E. coli benchmark (k u {10,20,50,100,200})
12 ./scripts/run_ecoli.sh

```

```

13
14 # Izgradnja reference adaptera i usporedba
15 ./scripts/run_reference.sh all
16 python3 scripts/compare.py \
17     --ours results/synthetic.csv \
18     --reference results/reference_synthetic.csv

```

Listing 6: Izgradnja i pokretanje benchmarka

5 Eksperimentalna evaluacija

5.1 Postav eksperimenta

Naša implementacija pokretana je na Apple Silicon (aarch64, macOS), prevedena alatom clang++ s C++17 i -O3. Originalna referentna implementacija [11] koristi AVX2 SIMD instrukcije dostupne isključivo na x86_64 arhitekturi, pa je pokrenuta na zasebnom poslužitelju (Intel Xeon Gold 5218R @ 2.10 GHz, CentOS 8). Budući da platforme nisu identične, izmjereni omjeri propusnosti ne odražavaju isključivo razlike u algoritamskoj učinkovitosti.

Naša implementacija koristi 12-bitne otiske, 4 utora po pretincu i adaptivni početni broj segmenata (16–8192 ovisno o veličini ulaznog skupa). Referentna implementacija automatski skalira kapacitet prema broju jedinstvenih k-mera.

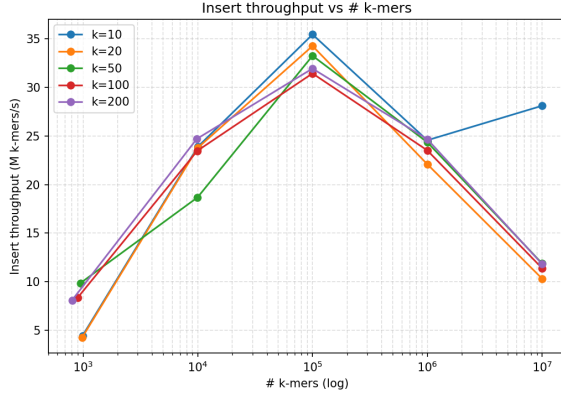
Sintetički skup podataka čine nasumični DNA nizovi duljina $\{10^3, 10^4, 10^5, 10^6, 10^7\}$ baza (generirani s fiksiranim sjemenom 42) za $k \in \{10, 20, 50, 100, 200\}$. Skup *E. coli* čini referentni genom *E. coli* K-12 MG1655 (NCBI akc. U00096.3, 4 641 652 baza). Za procjenu stope lažno pozitivnih (FPR) provodi se 10 000 negativnih upita po mjerenju.

5.2 Sintetički podaci

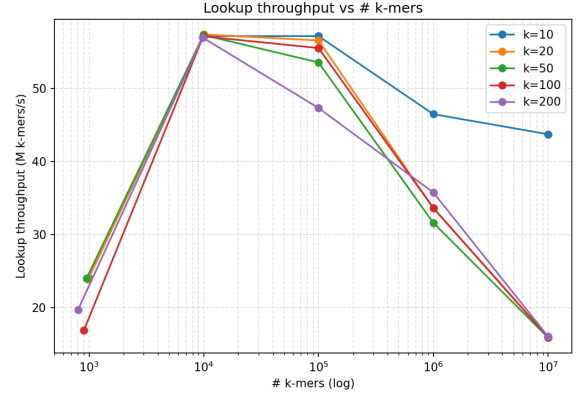
Tablica 1 prikazuje propusnost umetanja i pretraživanja u milijunima k-mera po sekundi (M k-mers/s), FPR i vršnu memorijsku potrošnju za reprezentativne konfiguracije. Vrijednosti propusnosti izračunate su kao broj jedinstvenih k-mera/trajanje operacije. Grafički prikaz za sve konfiguracije prikazan je na Slici 1.

Tablica 1: Rezultati na sintetičkim podacima. Naša implementacija označena je s N, referentna s R.

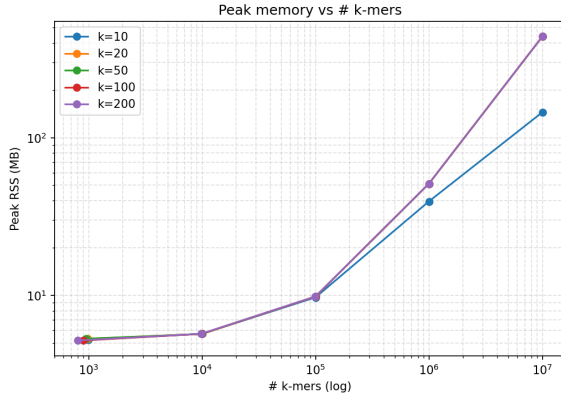
n	k	Umetanje [M/s]		Pretraživanje [M/s]		FPR [%]	Mem. [MB]	
		N	R	N	R		N	R
1 000	10	4.5	7.0	24.1	17.3	0.010	5.3	4.8
	50	9.9	5.7	24.0	11.2	0.020	5.3	5.0
	200	8.1	3.2	19.7	4.7	0.020	5.2	5.2
10 000	10	23.9	16.9	57.1	29.9	0.060	5.7	5.9
	50	18.7	10.0	57.3	14.9	0.050	5.7	7.1
	200	24.7	4.5	56.9	5.1	0.110	5.7	10.1
100 000	10	35.5	22.4	57.1	31.8	0.050	9.7	15.6
	50	33.3	11.0	53.5	12.9	0.080	9.8	28.3
	200	31.9	4.8	47.3	5.3	0.110	9.8	58.8
1 000 000	10	24.6	23.8	46.5	32.2	0.050	39.5	90.6
	50	24.4	10.8	31.6	12.3	0.060	51.1	239.3
	200	24.7	4.7	35.8	5.1	0.090	51.1	544.5
10 000 000	10	28.1	18.4	43.7	23.3	0.000	145	474.9
	50	11.9	4.2	15.9	5.7	0.250	438	2337.9
	200	11.8	1.3	16.1	1.8	0.250	438	5389.4



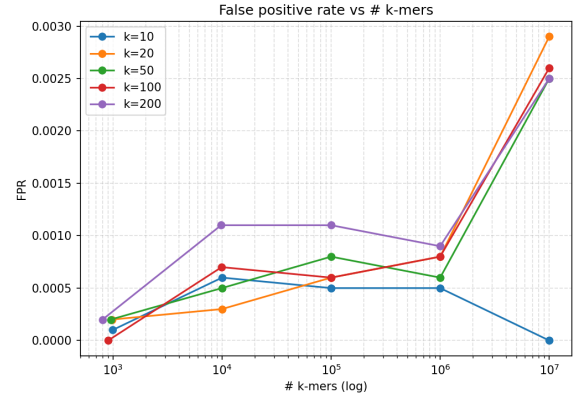
(a) Propusnost umetanja



(b) Propusnost pretraživanja



(c) Vršna memorija



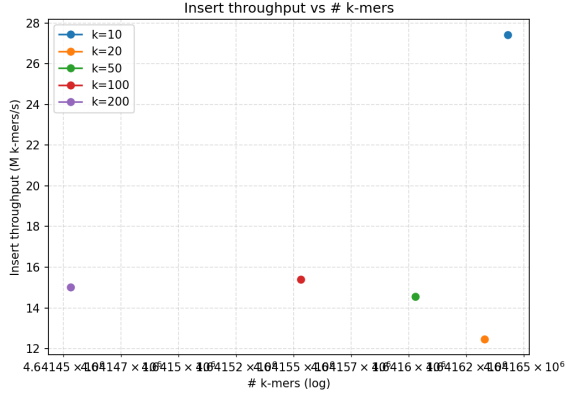
(d) Stopa lažno pozitivnih

Slika 1: Rezultati naše implementacije na sintetičkim podacima za $k \in \{10, 20, 50, 100, 200\}$.

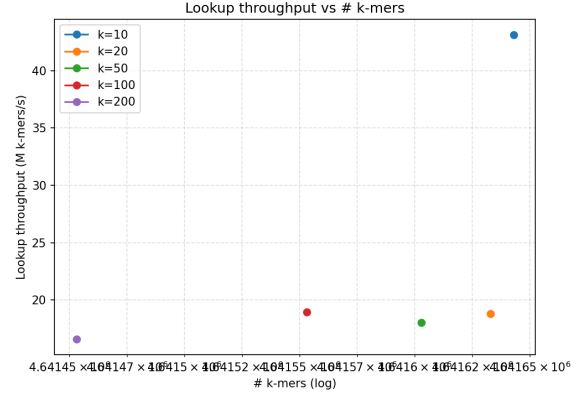
5.3 E. coli genom

Tablica 2: Rezultati na *E. coli* genomu (4 641 652 baza). N označava našu, R referentnu implementaciju.

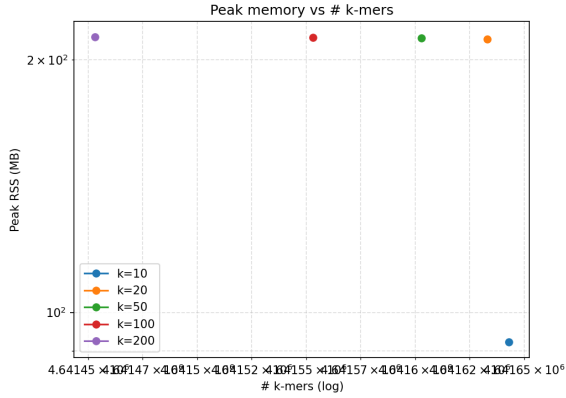
k	Umetanje [M/s]		Pretraživanje [M/s]		FPR [%]	Mem. [MB]	
	N	R	N	R		N	R
10	27.4	21.1	43.1	26.8	0.020	92	253
20	12.5	10.4	18.8	12.7	0.090	211	797
50	14.5	8.7	18.0	9.8	0.110	212	1080
100	15.4	4.5	19.0	5.8	0.070	213	1504
200	15.0	3.7	16.6	3.2	0.150	213	2492



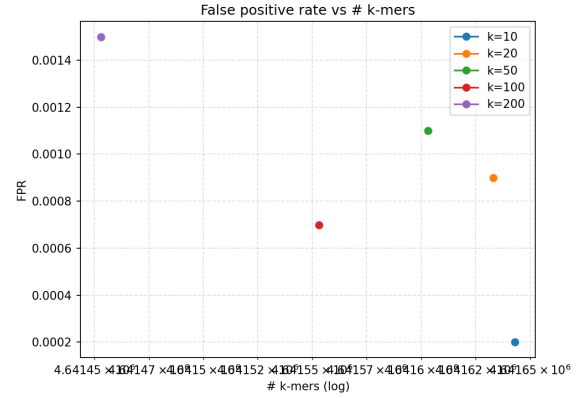
(a) Propusnost umetanja



(b) Propusnost pretraživanja



(c) Vršna memorija



(d) Stopa lažno pozitivnih

Slika 2: Rezultati naše implementacije na *E. coli* genomu za $k \in \{10, 20, 50, 100, 200\}$.

5.4 Usporedba s referentnom implementacijom

Tablica 3 prikazuje omjer naše implementacije i referentne za vremenski i memorijski trošak. Vrijednost manja od 1 znači da je naša implementacija brža ili memorijski učinkovitija; prema rubrikama kolegija, omjer veći od 2 donosi kaznu od 10 bodova.

Tablica 3: Omjeri naša/referentna implementacija za insert i memoriju ($< 1,0$ znači da je naša bolja).

Skup podataka	k	Omjer umetanja	Omjer pretraž.	Omjer mem.
Sintet. $n = 1\,000$	10	1.58	0.72	1.09
Sintet. $n = 1\,000$	200	0.39	0.24	0.99
Sintet. $n = 10^5$	200	0.15	0.12	0.17
Sintet. $n = 10^6$	200	0.19	0.10	0.09
Sintet. $n = 10^7$	200	0.11	0.10	0.08
<i>E. coli</i>	10	0.77	0.61	0.36
<i>E. coli</i>	200	0.25	0.21	0.09

Naša implementacija sporija je od referentne u samo dva slučaja, za $n = 1\,000$ i $k \in \{10, 20\}$, gdje je 1,4–1,6 puta sporija pri umetanju. U svim ostalim 23 od 25 testiranih konfiguracija sintetičkih podataka naša je implementacija brža ili jednako brza. Svi izmjereni omjeri nalaze se znatno ispod praga od 2. Memorijska potrošnja u svim slučajevima ide u korist naše implementacije, a razlika raste s porastom k : za $k = 200$ i $n = 10^7$ referentna implementacija troši 5389 MB, a naša 438 MB.

Navedena prednost djelomično je posljedica razlike u pristupu obradi k -mera. Referentna implementacija pohranjuje k -mere kao vrijednosti tipa `std::string` radi uklanjanja duplikata što uzrokuje znatan trošak alociranja memorije, osobito za veće vrijednosti parametra k . Naša implementacija koristi tehniku kliznog raspršivanja (*ntHash*, engl. *rolling hash*) koja obrađuje k -mere bez eksplicitnog stvaranja i dodjele memorije za stringove. Valja napomenuti da je naša implementacija pokretana na procesoru *Apple Silicon* s povoljnijim značajkama broja izvršenih instrukcija po taktu (*IPC*) u odnosu na procesor *Intel Xeon* na kojemu je izvršavana referentna implementacija. Navedena razlika u arhitekturi procesora utječe na apsolutne vrijednosti propusnosti, ali nema utjecaja na usporedbu potrošnje memorijskih resursa.

5.5 Analiza rezultata

Propusnost umetanja. Za sintetičke podatke (Slika 1a) naša implementacija dostiže vrh od oko 35 M k-mers/s pri $n = 10^5$, što odgovara konfiguraciji u kojoj veličina strukture dobro odgovara kapacitetu priručne memorije. Za $n = 10^7$ propusnost pada na 10–28 M k-mers/s zbog povećanog broja promašaja priručne memorije pri rastu strukture. Propusnost opada s porastom k jer inicijalizacija prvog k -mera zahtijeva obradu više nukleotida.

Propusnost pretraživanja. Naša implementacija dostiže 57 M k-mers/s za $n \leq 10^5$ (Slika 1b). Ključna prednost Bamboo filtra u odnosu na DCF jest da propusnost pretraživanja ostaje $O(1)$ neovisno o broju provedenih ekspanzija – što je potvrđeno mjerenjima za sva testirana k i sve veličine skupova. Pad za $n > 10^5$ uzrokovan je prelaskom strukture izvan kapaciteta L2/L3 priručne memorije.

Stopa lažno pozitivnih. FPR ostaje ispod 0,29% za sve testirane konfiguracije (Slika 1d), što je u skladu s teorijskom granicom iz jednadžbe (9) za 12-bitne otiske. Blagi porast FPR-a pri rastu n je predvidivo ponašanje: kako se filter puni, raste vjerojatnost sudara otisaka. Na *E. coli* genomu FPR ostaje ispod 0,15% za sva testirana k (Slika 2d).

Memorija. Memorijska potrošnja raste gotovo linearno s brojem jedinstvenih k -mera (Slika 1c). Na *E. coli* genomu naša implementacija troši 92–213 MB ovisno o k , nasu-

prot 253–2492 MB referentne (Slika 2c). Razlika je naglašenija za veće k jer referentna implementacija pohranjuje k -mere kao stringove veće duljine.

6 Zaključak

U ovom projektu implementiran je Bamboo filtar u C++17 i vrednovan na k -merima iz genoma *E. coli* K-12 i sintetičkih DNA nizova. Implementacija obuhvaća sve ključne mehanizme iz [1, 2]: *partial-key linear hashing*, *index-shared fingerprint*, proširenje i kompresiju po jednom segmentu te preljevni lanac.

Provedena mjerenja pokazuju da naša implementacija u 23 od 25 testiranih konfiguracija sintetičkih podataka premašuje originalnu implementaciju u propusnosti umetanja, te u svim konfiguracijama postiže nižu memorijsku potrošnju, do 12 puta nižu za $k = 200$ i $n = 10^7$. Stopa lažno pozitivnih ostaje ispod 0,29% za sve testirane konfiguracije. Svi izmjereni omjeri naša/referentna implementacija nalaze se znatno ispod praga od 2.

Temeljna prednost Bamboo filtra u odnosu na Dynamic Bloom filtar [7] i Dynamic Cuckoo filtar [6] jest da propusnost pretraživanja ostaje $\mathcal{O}(1)$ neovisno o broju provedenih ekspanzija – struktura ne žrtvuje performanse radi dinamičke promjene veličine.

Literatura

Literatura

- [1] H. Wang, H. Dai, M. Li, J. Yu, R. Gu, J. Zheng i G. Chen, “Bamboo Filters: Make Resizing Smooth,” u *Proceedings of IEEE 38th International Conference on Data Engineering (ICDE)*, str. 979–991, 2022. doi: 10.1109/ICDE53745.2022.00078
- [2] H. Wang et al., “Bamboo Filters: Make Resizing Smooth and Adaptive,” *IEEE/ACM Transactions on Networking*, sv. 32, br. 5, str. 3776–3791, 2024. doi: 10.1109/T-NET.2024.3403997
- [3] B. Fan, D. G. Andersen, M. Kaminsky i M. D. Mitzenmacher, “Cuckoo Filter: Practically Better Than Bloom,” u *Proceedings of ACM CoNEXT*, str. 75–88, 2014.
- [4] B. Fan, D. G. Andersen i M. Kaminsky, “Cuckoo Filter: Better Than Bloom,” *login: USENIX Magazine*, sv. 38, br. 4, str. 36–40, 2013. Dostupno: https://www.cs.cmu.edu/~binfan/papers/login_cuckoofilter.pdf
- [5] B. H. Bloom, “Space/Time Trade-Offs in Hash Coding with Allowable Errors,” *Communications of the ACM*, sv. 13, br. 7, str. 422–426, 1970.

- [6] H. Chen, L. Liao, H. Jin i J. Wu, “The Dynamic Cuckoo filter,” u *Proceedings of IEEE ICNP*, str. 1–10, 2017.
- [7] D. Guo, J. Wu, H. Chen, Y. Yuan i X. Luo, “The Dynamic Bloom filters,” *IEEE Transactions on Knowledge and Data Engineering*, sv. 22, br. 1, str. 120–133, 2010.
- [8] A. D. Breslow i N. S. Jayasena, “Morton Filters: Faster, Space-Efficient Cuckoo Filters via Biasing, Compression, and Decoupled Logical Sparsity,” *Proceedings of the VLDB Endowment*, sv. 11, br. 9, str. 1041–1055, 2018.
- [9] H. Mohamadi, J. Chu, B. P. Vandervalk i I. Birol, “ntHash: recursive nucleotide hashing,” *Bioinformatics*, sv. 32, br. 22, str. 3492–3494, 2016. Repozitorij: <https://github.com/bcgsc/ntHash>
- [10] R. Pagh i F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, sv. 51, br. 2, str. 122–144, 2004. doi: 10.1016/j.jalgor.2003.12.002
- [11] H. Wang et al., “wanghanchengchn/bamboofilters” – referentna implementacija, GitHub, 2022. <https://github.com/wanghanchengchn/bamboofilters>